

**LAPORAN PROYEK PROFESIONAL
JALUR AKADEMISI**

**IDENTIFIKASI PENYAKIT DAUN PADA TANAMAN *SOLANACEAE*
DAN *ROSACEAE* MENGGUNAKAN *DEEP LEARNING***



ALLAN BIL FAQIH

5210411383

KONSENTRASI : SISTEM CERDAS

SEMESTER GANJIL TAHUN AKADEMIK 2024/2025

PROGRAM STUDI INFORMATIKA

FAKULTAS SAINS & TEKNOLOGI

UNIVERSITAS TEKNOLOGI YOGYAKARTA

HALAMAN PENGESAHAN

**LAPORAN PROYEK PROFESIONAL
JALUR AKADEMISI**

**IDENTIFIKASI PENYAKIT DAUN PADA TANAMAN *SOLANACEAE*
DAN *ROSACEAE* MENGGUNAKAN *DEEP LEARNING***

disusun oleh

ALLAN BIL FAQIH

5210411383

Laporan ini telah disahkan oleh pembimbing
pada tanggal 21 Desember 2024

Dosen Pembimbing Proyek Profesional



Dr. Donny Avianto, S.T., M.T.

NIK. 11 1015 077

DAFTAR ISI

HALAMAN PENGESAHAN.....	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR	iii
DAFTAR TABEL.....	v
BAB I RIWAYAT PUBLIKASI ARTIKEL	1
1.1 Bukti Pengiriman Artikel	1
1.2 Lini Masa dan Status Artikel	3
1.3 Poin Revisi dan Hasil Perbaikan Artikel	3
BAB II PENAMBAHAN FITUR SISTEM.....	21
2.1 Target Penambahan Fitur Sistem.....	21
2.2 Perbandingan Fitur Sistem	22
BAB III PENGENALAN ANTAR MUKA SISTEM	23
3.1 Halaman <i>Splash Screen</i>	23
3.2 Halaman Utama	24
3.3 Halaman Pengambilan Gambar	28
3.4 Halaman <i>Cropping</i> Gambar	30
3.5 Halaman Hasil	31
3.6 Halaman Riwayat	35
BAB IV PROSEDUR PENGGUNAAN SISTEM	40
4.1 Prosedur Fitur Identifikasi.....	40
4.2 Prosedur Fitur Riwayat.....	44

DAFTAR GAMBAR

Gambar 1.1 Persetujuan Submit ke Jurnal	1
Gambar 1.2 Bukti Pengiriman Naskah ke Jurnal Tujuan	2
Gambar 2.1 Usulan Penambahan Fitur pada Poster Seminar PUI	21
Gambar 2.2 Target Penambahan Fitur Sistem dari Google Form	22
Gambar 3.1 Tampilan Halaman <i>Splash Screen</i>	23
Gambar 3.2 Kode Program Halaman <i>Splash Screen</i>	24
Gambar 3.3 Tampilan Halaman Utama.....	25
Gambar 3.4 Kode Program untuk Tampilan Halaman Utama	26
Gambar 3.5 Kode Program Tombol Menuju Halaman Riwayat	26
Gambar 3.6 Kode Program Pemrosesan Gambar	27
Gambar 3.7 Kode Program Pengiriman Gambar.....	27
Gambar 3.8 Kode Program <i>API Service</i>	28
Gambar 3.9 Kode Program Tombol Menuju Halaman Hasil	28
Gambar 3.10 Tampilan Halaman Kamera dan Galeri	29
Gambar 3.11 Kode Program untuk Membuka Kamera dan Galeri	29
Gambar 3.12 Tampilan Halaman <i>Cropping</i> Gambar	30
Gambar 3.13 Kode Program untuk Fungsi <i>Cropping</i> Gambar.....	31
Gambar 3.14 Tampilan Halaman Hasil Identifikasi	31
Gambar 3.15 Kode Program Tampilan Halaman Hasil.....	32
Gambar 3.16 Kode Program Penyimpanan Riwayat Identifikasi	32
Gambar 3.17 Kode Program Inisialisasi Halaman Hasil	33
Gambar 3.18 Kode Program untuk Menampilkan Gambar Hasil	33
Gambar 3.19 Kode Program untuk Membuat Tautan Teks	34
Gambar 3.20 Kode Program untuk Mengambil Informasi Penyakit.....	34
Gambar 3.21 Tampilan Halaman Riwayat Identifikasi	35
Gambar 3.22 Kode Program Tampilan Utama Halaman Riwayat	36
Gambar 3.23 Kode Program Tampilan Item Riwayat	37
Gambar 3.24 Kode Program untuk Memuat Item Riwayat.....	37
Gambar 3.25 Kode Program Inisialisasi Halaman Riwayat	38

Gambar 3.26	Kode Program untuk Menghapus Item Riwayat dari Database.....	38
Gambar 3.27	Kode Program untuk Menampilkan Konfirmasi Penghapusan	38
Gambar 3.28	Kode Program untuk Menghapus Item Riwayat dari Halaman.....	39
Gambar 3.29	Kode Program untuk Menampilkan Informasi Penyakit	39

DAFTAR TABEL

Tabel 1.1 Rangkuman Riwayat Pengiriman Artikel.....	3
Tabel 1.2 Rangkuman Revisi <i>Reviewer</i> Ronde 1	4
Tabel 1.3 Rangkuman Revisi <i>Reviewer-1</i> Ronde 2.....	6
Tabel 1.4 Rangkuman Revisi <i>Reviewer-2</i> Ronde 2.....	11
Tabel 2.1 Perbandingan Fitur Sistem	22

BAB I

RIWAYAT PUBLIKASI ARTIKEL

1.1 Bukti Pengiriman Artikel

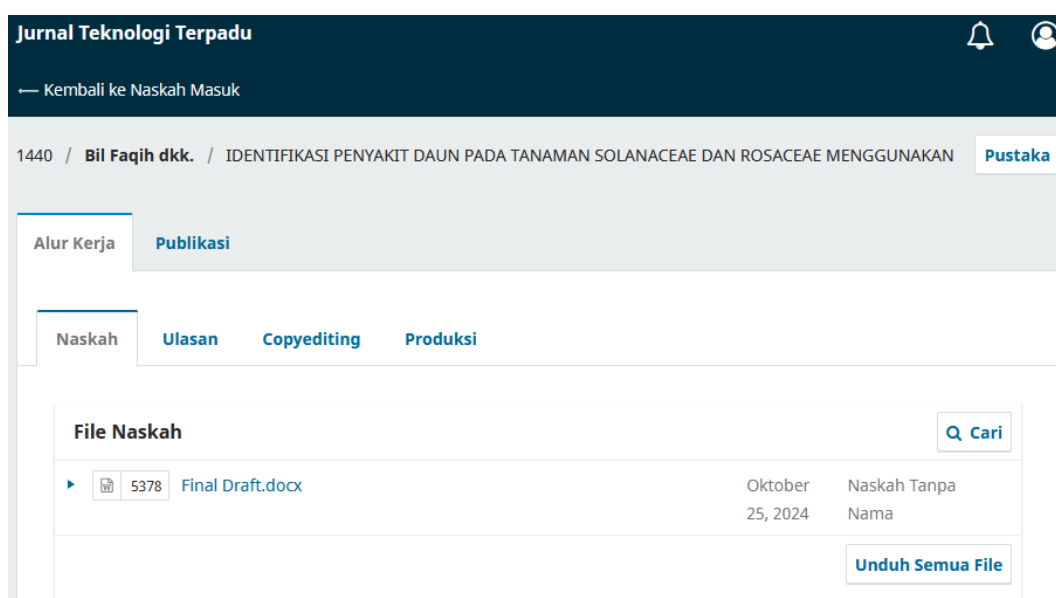
Penelitian yang dilakukan dalam mata kuliah proyek profesional telah diajukan untuk publikasi di Jurnal Teknologi Terpadu, sebuah jurnal yang memiliki akreditasi Sinta 4 (akreditasi dapat diverifikasi melalui <https://sinta.kemdikbud.go.id/journals/profile/3716>). Pengajuan artikel ke jurnal dilakukan setelah melalui proses konsultasi dan mendapatkan persetujuan dari dosen pembimbing proyek profesional. Seperti yang ditunjukkan pada Gambar 1.1, dosen pembimbing memberikan izin untuk mengirimkan artikel tersebut ke jurnal pada 25 Oktober 2024.



Gambar 1.1 Persetujuan Submit ke Jurnal

Setelah memperoleh izin, artikel dikirimkan ke Jurnal Teknologi Terpadu dengan mengakses portal submission di <https://journal.nurulfikri.ac.id/index.php/jtt/about/submissions>. Pengiriman artikel tersebut telah

didokumentasikan dan dapat dilihat pada Gambar 1.2. Perlu dicatat bahwa pada saat pengajuan untuk persetujuan (Gambar 1.1), artikel ini masih menggunakan judul yang tercantum pada gambar tersebut. Namun, pada tahap revisi ronde kedua setelah pengiriman artikel, judul penelitian mengalami perubahan sesuai dengan masukan dari reviewer jurnal. Perubahan ini dirangkum dalam Tabel 1.4, yang memperlihatkan komentar reviewer serta perbandingan judul sebelum dan sesudah perbaikan.



Gambar 1.2 Bukti Pengiriman Naskah ke Jurnal Tujuan

Pengiriman naskah dilaksanakan pada 25 Oktober 2024 seussai mendapatkan persetujuan dari dosen pembimbing proyek profesional, sebagaimana terlihat pada Gambar 1.2. Naskah yang diajukan juga telah melalui proses pemeriksaan plagiarisme menggunakan aplikasi Turnitin. Hasil pemeriksaan menunjukkan tingkat kemiripan (*similarity index*) sebesar 12% secara keseluruhan, dengan persentase kesamaan untuk setiap bagian tidak melebihi 1-2%.

1.2 Lini Masa dan Status Artikel

Pada bagian berikut, terdapat rangkuman riwayat pengiriman artikel ke Jurnal Teknologi Terpadu beserta status terbaru artikel milik peneliti, yang dapat dilihat pada Tabel 1.1.

Tabel 1.1 Rangkuman Riwayat Pengiriman Artikel

Tanggal	Aktivitas
25 Oktober 2024	Dosen pengampu Proyek Profesional memberikan persetujuan untuk mengirimkan naskah, yang kemudian langsung disubmit ke Jurnal Teknologi Terpadu.
8 November 2024	Peneliti menerima hasil review dari satu reviewer mitra bersari (Ronde #1), melakukan revisi berdasarkan masukan tersebut, dan mengirimkan kembali naskah yang telah direvisi.
28 November 2024	Peneliti mendapat keputusan bahwa artikel diterima dengan catatan revisi, di mana dua reviewer mitra bersari memberikan hasil review mereka (Ronde #2).
28 November – 1 Desember 2024	Peneliti menyelesaikan revisi sesuai hasil review dari Ronde #2.
2 Desember 2024	Naskah hasil revisi Ronde #2 dikirimkan kembali, bersamaan dengan proses pembayaran.
4 Desember 2024	Peneliti menerima <i>Letter of Acceptance</i> .
24 Desember 2024	Artikel telah dipublikasikan dan dapat diakses melalui tautan berikut: https://journal.nurulfikri.ac.id/index.php/JTT/article/view/1440/377

1.3 Poin Revisi dan Hasil Perbaikan Artikel

Selama proses pengiriman artikel ke Jurnal Teknologi Terpadu, peneliti melalui dua tahap revisi. Pada Ronde 1, peneliti menerima masukan dari satu *reviewer*, di mana *reviewer* tersebut memberikan 4 poin revisi. Rincian poin revisi

beserta perbaikannya telah dirangkum oleh peneliti dalam bentuk tabel. Tabel 1.2 menyajikan rangkuman revisi dari *reviewer* ke-1.

Tabel 1.2 Rangkuman Revisi *Reviewer* Ronde 1

Poin Revisi No.1	Bab: Nama Penulis
<u>Komentar reviewer:</u> “Mohon lengkapi nama, afiliasi, dan email dari seluruh penulis.”	
<u>Sebelum Perbaikan:</u> Penulis¹, Penulis² ^{1,2} Prodi, Universitas Alamat, Alamat, Indonesia Kode Pos email@email.com, email2@email.com	
<u>Sesudah Perbaikan:</u> Allan Bil Faqih¹, Donny Avianto² ^{1,2} Informatika, Universitas Teknologi Yogyakarta Yogyakarta, D.I Yogyakarta, Indonesia 55164 allanbilfaqih214@gmail.com, donny@uty.ac.id	
...	
Poin Revisi No.2	Bab: Format Artikel
<u>Komentar reviewer:</u> “Perhatikan kembali jarak antar baris dan paragraf. Pastikan sesuai dengan format template.”	
<u>Sebelum Perbaikan:</u> [1] PENDAHULUAN [2] METODE PENELITIAN [3] HASIL DAN PEMBAHASAN [4] KESIMPULAN ----- 3.2 Pengujian Model Pengujian berfungsi untuk menentukan keefektifan model dalam mengidentifikasi objek ...	
<u>Sesudah Perbaikan:</u> 1. PENDAHULUAN 2. METODE PENELITIAN	

3. HASIL DAN PEMBAHASAN	
4. KESIMPULAN	

3.2 Pengujian Model	
Pengujian berfungsi untuk menentukan keefektifan model dalam mengidentifikasi objek ...	
...	
Poin Revisi No.3	Bab: Kata Kunci
<u>Komentar reviewer:</u> “Harap urut kata kunci sesuai abjad”	
<u>Sebelum Perbaikan:</u> Keywords: <i>deep learning, plant disease detection, convolutional neural networks, transfer learning, computer vision</i> Kata kunci: <i>deep learning, deteksi penyakit tanaman, convolutional neural networks, transfer learning, computer vision</i>	
<u>Sesudah Perbaikan:</u> Keywords: <i>Computer vision, convolutional neural networks, deep learning, plant disease detection, transfer learning</i> Kata kunci: <i>Computer vision, convolutional neural networks, deep learning, deteksi penyakit tanaman, transfer learning</i>	
...	
Poin Revisi No.4	Bab: Metode Penelitian
<u>Komentar reviewer:</u> "Mohon untuk perbaiki penyebutan kata "Table" menjadi "Tabel"."	
<u>Sebelum Perbaikan:</u> - ... terlihat pada Table 1 berikut ... - Table 1. Dataset yang dimodifikasi - seperti yang terlihat pada Table 2 di bawah ...	
<u>Sesudah Perbaikan:</u> - ... terlihat pada Tabel 1 berikut ... - Tabel 1. Dataset yang dimodifikasi - seperti yang terlihat pada Tabel 2 di bawah ...	

Selanjutnya, pada tahap revisi Ronde 2, peneliti menerima masukan dari dua reviewer. Jumlah revisi yang diberikan cukup signifikan, dengan reviewer ke-1 memberikan 2 poin dan reviewer ke-2 memberikan 8 poin. Rincian poin revisi beserta hasil perbaikannya telah dirangkum dalam bentuk tabel. Tabel 1.3 menyajikan rangkuman revisi dari reviewer ke-1, sementara Tabel 1.4 menyajikan rangkuman revisi dari reviewer ke-2.

Tabel 1.3 Rangkuman Revisi *Reviewer-1* Ronde 2

Poin Revisi No.1	Bab: Pendahuluan
<u>Komentar reviewer:</u> “Kontribusi utama penelitian untuk dapat diperjelas dan dipertegas, kembali?”	
<u>Sebelum Perbaikan:</u> - Bab 1: ... Penelitian ini menggunakan dataset <i>PlantVillage</i> , yang terdiri dari 54.306 citra daun dari 14 spesies tanaman yang berbeda, termasuk 38 kategori daun yang sakit dan sehat [8]. ... Berbeda dengan penelitian sebelumnya yang umumnya terbatas pada satu spesies tanaman [8], [9], [10], [11], [12], [13], penelitian ini bertujuan mengembangkan model yang lebih adaptif dengan membandingkan kinerja <i>Transfer Learning</i> menggunakan <i>InceptionV3</i> dan CNN yang dirancang khusus. Pendekatan ini diharapkan dapat menghasilkan solusi yang lebih serbaguna dan mudah diterapkan dalam praktik pertanian di dunia nyata, sekaligus mengatasi keterbatasan generalisasi yang ditemukan pada penelitian-penelitian sebelumnya.	
<u>Sesudah Perbaikan:</u> - Bab 1: ... Penelitian ini menggunakan dataset <i>PlantVillage</i> , yang terdiri dari 54.306 citra daun yang memiliki format JPEG dari 14 spesies tanaman yang berbeda, termasuk 38 kategori daun yang sakit dan sehat [8]. ... Kontribusi utama penelitian ini adalah pengembangan dan mengevaluasi arsitektur <i>deep learning</i> untuk deteksi penyakit pada tanaman dari keluarga <i>Solanaceae</i> dan <i>Rosaceae</i> , dengan membandingkan dua pendekatan utama: <i>Transfer Learning</i> menggunakan <i>InceptionV3</i> dan CNN yang dirancang khusus. Berbeda dari penelitian sebelumnya yang umumnya terbatas pada deteksi penyakit untuk satu jenis tanaman saja [8], [9], [10], [11], [12], [13], melalui pendekatan multi-tanaman, penelitian ini tidak hanya menawarkan solusi praktis	

bagi petani yang umumnya menanam beragam jenis tanaman secara bersamaan, tetapi juga memberikan wawasan mendalam tentang keunggulan relatif dari setiap arsitektur melalui evaluasi dan perbandingan kedua pendekatan tersebut. Hasil evaluasi diharapkan memberikan solusi yang lebih serbaguna untuk praktik pertanian dunia nyata, sekaligus mengatasi keterbatasan generalisasi dari penelitian-penelitian sebelumnya.

Poin Revisi No.2	Bab: Metode Penelitian
-------------------------	-------------------------------

Komentar reviewer:

“Anda perlu menjelaskan bagaimana state of the art yang dipaparkan itu hasil penelitiannya, selain itu bagaiman denagn kelemahan dari riset tersebut sehingga mengerucut pada artikel ini diusulkan model CNN yang diusulkan?”

Sebelum Perbaikan:

- Bab 2:

...

Henry et al. menggunakan CNN untuk klasifikasi penyakit pada daun tomat dengan dataset sebanyak 18.162 citra dan mencapai akurasi *training* 94,06% dengan *loss function* 7,8% [8]. Rijal et al. mengimplementasikan CNN untuk mengklasifikasikan penyakit pada Padi dengan tingkat akurasi validasi yang mencapai 80% [13]. Sulistiyana dan Anardani mengombinasikan metode CNN dengan *Support Vector Machine* (SVM) untuk mendeteksi penyakit tanaman jagung, dengan hasil akurasi CNN mencapai 98% dibandingkan SVM 87% [10]. Sementara itu, Pratama et al. menggunakan CNN dengan *transfer learning* untuk klasifikasi penyakit daun pisang dan mencapai akurasi 92% setelah mengatasi masalah *overfitting* menggunakan *dropout* [12].

...

Berdasarkan kajian pustaka di atas, dapat disimpulkan bahwa CNN dengan berbagai arsitekturnya telah terbukti efektif dalam klasifikasi penyakit tanaman melalui citra daun. Namun, sebagian besar penelitian masih terfokus pada satu jenis tanaman spesifik. berbagai jenis tanaman, dengan harapan dapat memberikan solusi yang lebih komprehensif dan mudah diakses bagi petani dalam mendeteksi penyakit tanaman secara cepat dan akurat.

Sesudah Perbaikan:

- Bab 2:

...

Henry et al. mengembangkan model CNN yang menjadi *state of the art* untuk klasifikasi penyakit pada daun tomat dengan dataset sebanyak 18.162 citra dan mencapai akurasi *training* 94,06% dengan *loss function* 7,8% [8]. Sulistiyana dan

Anardani memberikan kontribusi penting dengan membuktikan superioritas CNN (98%) dibanding SVM (87%) dalam deteksi penyakit jagung, menunjukkan bahwa CNN lebih efektif dalam mengekstrak fitur kompleks dari citra daun [10]. Rijal et al. juga menunjukkan efektivitas CNN dalam mengklasifikasikan penyakit pada Padi dengan tingkat akurasi validasi mencapai 80% [13].

...

Sementara itu, Pratama et al. menggunakan model *transfer learning* CNN-nya menghadirkan inovasi dengan mengatasi masalah *overfitting* menggunakan *dropout* pada klasifikasi penyakit daun pisang, mencapai akurasi 92% yang stabil bahkan dengan dataset terbatas [12].

Meskipun penelitian-penelitian tersebut menunjukkan hasil yang menjanjikan, terdapat beberapa keterbatasan signifikan. Pendekatan *single-crop* yang digunakan membatasi aplikasi praktis di lapangan. Model-model ini memerlukan pelatihan ulang untuk setiap jenis tanaman baru, sehingga kurang efisien untuk implementasi skala besar. Selain itu, kemampuan generalisasi model terhadap berbagai jenis tanaman belum teruji secara menyeluruh.

Berdasarkan analisis *state of the art* dan keterbatasan tersebut, penelitian ini mengusulkan penggunaan model yang dapat menangani *multiple crops* sekaligus. Pendekatan ini diharapkan dapat mempertahankan akurasi tinggi yang telah dicapai penelitian sebelumnya, sambil mengatasi keterbatasan dalam hal generalisasi dan implementasi praktis. Selain itu, akan dilakukan perbandingan beberapa arsitektur *deep learning* untuk mengidentifikasi model yang paling optimal dalam menangani kasus *multiple crops*. Dengan fokus pada tanaman dari keluarga *Solanaceae* dan *Rosaceae*, penelitian ini bertujuan mengembangkan solusi yang lebih komprehensif dan mudah diakses bagi petani dalam mendeteksi penyakit tanaman secara cepat dan akurat.

Poin Revisi No.3

Bab: Metode Penelitian & Hasil dan Pembahasan & Daftar Pustaka

Komentar reviewer:

“Dari semua hasil eksperimen yang dilakukan, bagaimana dengan hasil performance tingkat akurasi yang dihasilkan? Bagaimana setiap model yang diusulkan dengan parameter yang digunakan (belum jelas darimana hasil nilai tersebut?)”

Sebelum Perbaikan:

- Bab 2:

...

Pada model CNN *Deep Feature Extractor* (DFE), arsitektur lapisan Keras dari *InceptionV3* disederhanakan dengan mengganti lapisan-lapisan tersebut dengan

serangkaian lapisan konvolusi dan pooling. Setiap lapisan konvolusi diikuti oleh aktivasi *ReLU*, yang berfungsi untuk memperkenalkan *non-linearitas*, sebelum diakhiri dengan lapisan *Flatten* untuk mengubah data tiga dimensi menjadi satu dimensi, sehingga memudahkan proses klasifikasi. Arsitektur DFE dirancang untuk menangkap fitur-fitur penting dari citra input dengan efisien, sebagaimana terlihat pada Tabel 4 berikut.

...untuk mencegah overfitting. Regularisasi L2 diterapkan pada lapisan Dense untuk memberikan ...

- Bab 3

...

Setelah parameter dasar ditetapkan, eksperimen dapat dimulai. Proses pelatihan akan dilakukan dengan menggunakan *Kaggle IDE* karena kecepatan, keefektifan biaya, dan kemudahan penggunaannya. Hasil eksperimen dapat dilihat pada Tabel 7 berikut, dengan waktu pelatihan yang memakan sekitar 1 jam untuk setiap model berbasis *InceptionV3* dan sekitar 30 menit untuk setiap model kedua jenis CNN kustom.

...

Di antara 12 eksperimen yang dilakukan, terdapat 3 eksperimen yang di-*highlight*, yaitu eksperimen yang mewakili 3 model CNN yang digunakan: Model *InceptionV3*, CNN DFE, dan LCNNs. Eksperimen yang terpilih adalah eksperimen 1 (*InceptionV3*), 5 (CNN DFE), dan 9 (LCNN), dengan akurasi pelatihan masing-masing adalah 98%, 94%, dan 99%. Untuk menggambarkan lebih lanjut kinerja mereka, kurva akurasi untuk hasil ini ditampilkan pada Gambar 6 berikut ...

- Daftar Pustaka

...

[21] I. O. Muraina, "Ideal Dataset Splitting Ratios In Machine Learning Algorithms: General Concerns For Data Scientists And Data Analysts," in *7th International Mardin Artuklu Scientific Researches Conference*, Mardin: Mardin Artuklu Kongresi, Feb. 2022, pp. 496–504.

Sesudah Perbaikan:

- Bab 2:

...

Pada model CNN *Deep Feature Extractor* (DFE), arsitektur lapisan *Keras* dari *InceptionV3* disederhanakan dengan mengganti lapisan-lapisan tersebut dengan rangkaian lapisan konvolusi dan *pooling*. Jumlah filter konvolusi (32 dan 64)

dipilih berdasarkan hasil penelitian sebelumnya yang menunjukkan bahwa nilai ini optimal untuk mendeteksi fitur pada citra tanaman. Ukuran *dense layer* sebesar 256 juga ditentukan dengan merujuk pada penelitian serupa, yang menyatakan bahwa nilai ini memberikan keseimbangan antara kapasitas model dan efisiensi komputasi. Setiap lapisan konvolusi diikuti oleh aktivasi *ReLU*, yang berfungsi untuk memperkenalkan *non-linearitas*, sebelum diakhiri dengan lapisan *Flatten* untuk mengubah data tiga dimensi menjadi satu dimensi, sehingga memudahkan proses klasifikasi. Arsitektur DFE dirancang untuk menangkap fitur-fitur penting dari citra input dengan efisien, sebagaimana terlihat pada Tabel 4 berikut.

... Dense yang lebih beragam dan penerapan dropout yang lebih tinggi untuk mencegah overfitting. Penggunaan 128 filter pada *conv2d_2* merupakan peningkatan dari layer sebelumnya (64) untuk memungkinkan deteksi fitur yang lebih kompleks pada level yang lebih dalam. Regularisasi L2 diterapkan pada lapisan Dense untuk memberikan ...

- Bab 3

...

Parameter dasar dalam penelitian ini dipilih berdasarkan praktik terbaik dan hasil penelitian sebelumnya untuk memastikan efisiensi dan performa model. Ukuran input 299x299 piksel digunakan untuk *InceptionV3* sesuai spesifikasinya [22], sementara ukuran 224x224 untuk CNN kustom dipilih karena efisien dalam menangkap fitur dan komputasi [23]. *Batch size* bervariasi (32, 64, 40) untuk mengoptimalkan stabilitas pelatihan dan pemanfaatan memori. Data augmentasi mencakup *rescaling* (1/255) untuk normalisasi, rotasi 40°, pergeseran posisi (0.2), serta *shear* dan *zoom* (0.2) untuk meningkatkan tingkat *robustness* pada model terhadap variasi data.

Base learning rate 0.001 dan fungsi *loss categorical_crossentropy* dipilih untuk stabilitas pelatihan pada masalah klasifikasi multikelas, dengan *epochs* maksimum 20 dan *early stopping (patience 3)* untuk mencegah *overfitting*. *Callback* seperti reduksi *learning rate* (faktor 0.5) dan momentum 0.99 digunakan untuk mempercepat konvergensi. Epsilon 0.001 pada *batch normalization* menjaga stabilitas numerik selama pelatihan. Parameter ini dipilih untuk memberikan keseimbangan optimal antara kapasitas model dan efisiensi komputasi.

Setelah parameter dasar ditetapkan, eksperimen dilaksanakan. Proses pelatihan dilakukan menggunakan *Kaggle IDE* karena memiliki kecepatan tinggi, biaya yang efektif, dan kemudahan penggunaan. Hasil eksperimen ditunjukkan pada

Tabel 7, dengan durasi pelatihan sekitar 1 jam untuk setiap model berbasis *InceptionV3* dan sekitar 30 menit untuk setiap model CNN kustom.

...

Hasil eksperimen diperoleh dari pengujian berbagai konfigurasi model sesuai pada tabel 6 pada *Kaggle IDE*. Dari tabel hasil, terlihat bahwa model *InceptionV3* dan LCNN menunjukkan performa paling konsisten dengan akurasi tinggi (98-99%) dan loss rendah (0.2-0.4), sementara model CNN DFE memiliki performa sedikit lebih rendah dengan akurasi 80-94%.

Maka dari itu dari antara 12 eksperimen yang dilakukan, 3 eksperimen yang memiliki akurasi tertinggi di setiap jenis model diambil, yaitu eksperimen yang mewakili 3 model CNN yang digunakan: Model *InceptionV3*, CNN DFE, dan LCNNs. Eksperimen yang terpilih adalah eksperimen di-*highlight* pada tabel diatas yaitu 1, 5, dan 9. Untuk menggambarkan lebih lanjut kinerja mereka, kurva akurasi untuk hasil ini ditampilkan pada Gambar 6 berikut.

- Daftar Pustaka

...

[21] I. O. Muraina, "Ideal Dataset Splitting Ratios In Machine Learning Algorithms: General Concerns For Data Scientists And Data Analysts," in *7th International Mardin Artuklu Scientific Researches Conference*, Mardin: Mardin Artuklu Kongresi, Feb. 2022, pp. 496–504.

[22] S. Ramaneswaran, K. Srinivasan, P. M. D. R. Vincent, and C.-Y. Chang, "Hybrid Inception v3 XGBoost Model for Acute Lymphoblastic Leukemia Classification," *Computational and Mathematical Methods in Medicine*, vol. 2021, pp. 1–10, Jul. 2021, doi: 10.1155/2021/2577375.

[23] M. L. Richter, W. Byttner, U. Krumnack, A. Wiedenroth, L. Schallner, and J. Shenk, "(Input) Size Matters for CNN Classifiers," 2021, pp. 133–144. doi: 10.1007/978-3-030-86340-1_11.

Tabel 1.4 Rangkuman Revisi *Reviewer-2* Ronde 2

Poin Revisi No.1	Bab: Judul
<u>Komentar reviewer:</u> “lebih baik metodenya diganti dengan deep learning karena dilihat bukan hanya kepada inceptionV3 saja dalam pembahasannya”	
<u>Sebelum Perbaikan:</u> Identifikasi Penyakit Daun Pada Tanaman Solanaceae Dan Rosaceae Menggunakan InceptionV3	

<u>Sesudah Perbaikan:</u> Identifikasi Penyakit Daun Pada Tanaman Solanaceae Dan Rosaceae Menggunakan Deep Learning	
Poin Revisi No.2	Bab: Abstrak
<u>Komentar reviewer:</u> “Abstrak belum ada permasalahan yang ditulis, data yang digunakan apakah berupa image yang jpg atau bmp dan didapat dari mana? Serta, data atau image yang mau diolah itu belum ada dijelaskan dalam penelitian ini”	
<u>Sebelum Perbaikan:</u> Abstract <i>With a global population projection of 9.7 billion by 2050, agriculture faces significant challenges in ensuring food security. One of the main obstacles is plant diseases that can reduce crop yields by up to 40% per year. This research aims to develop a deep learning-based plant disease detection system using Convolutional Neural Networks (CNN) applied to two plant families, Solanaceae and Rosaceae. The dataset used is PlantVillage with 54,306 leaf images. Three deep learning models were developed: one using transfer learning with InceptionV3 architecture and two custom CNNs (DFE and LCNN). The LCNN model demonstrated superior performance, achieving 99% training and validation accuracy and 95% testing accuracy, compared to InceptionV3 with 96% training, 98% validation, and 92% testing accuracy, and DFE with 86% training, 94% validation, and 82% testing accuracy. This highlights the potential of task-specific architectures over complex models for plant disease detection. Confusion matrix analysis revealed challenges in distinguishing between healthy potato and late blight-infected potato leaves, as well as cedar apple rust. Suggestions for future development include implementing the model in a mobile application with client-server architecture, in order to provide farmers with an easily accessible tool for rapid plant disease detection.</i>	
Abstrak Dengan proyeksi populasi global sebesar 9,7 miliar pada tahun 2050, pertanian menghadapi tantangan yang signifikan dalam memastikan ketahanan pangan. Salah satu kendala utama adalah penyakit tanaman yang dapat menurunkan hasil panen hingga 40% per tahun. Penelitian ini bertujuan untuk mengembangkan sistem deteksi penyakit tanaman berbasis <i>deep learning</i> menggunakan <i>Convolutional Neural Networks</i> (CNN) yang diterapkan pada dua famili	

tanaman, *Solanaceae* dan *Rosaceae*. Dataset yang digunakan adalah *PlantVillage* dengan 54.306 citra daun. Tiga model *deep learning* dikembangkan: satu model yang menggunakan *transfer learning* dengan arsitektur *InceptionV3* dan dua CNN kustom (DFE dan LCNN). Model LCNN menunjukkan kinerja yang unggul, mencapai 99% akurasi pelatihan dan validasi serta 95% akurasi pengujian, dibandingkan dengan *InceptionV3* dengan 96% pelatihan, 98% validasi, dan 92% akurasi pengujian, serta DFE dengan 86% pelatihan, 94% validasi, dan 82% akurasi pengujian. Hal ini menunjukkan potensi arsitektur yang spesifik dibandingkan model yang kompleks untuk deteksi penyakit tanaman. Analisis *confusion matrix* mengungkapkan tantangan dalam membedakan antara kentang sehat dan daun kentang yang terinfeksi *late blight*, serta *cedar apple rust*. Saran pengembangan ke depan adalah implementasi model dalam aplikasi *mobile* dengan arsitektur *client-server*, agar dapat menyediakan alat yang mudah diakses bagi petani untuk deteksi penyakit tanaman secara cepat.

Sesudah Perbaikan:

Abstract

With a projected global population of 9.7 billion by 2050, agriculture faces significant challenges in ensuring food security. One major obstacle is plant diseases that reduce crop yields by 40% per year. Previous research is often limited to disease detection in a single plant species, thus poorly reflecting multi-species needs in real agricultural practices. This research aims to develop and evaluate deep learning-based plant disease detection system using Convolutional Neural Networks (CNN) applied to two plant families, Solanaceae and Rosaceae. The dataset used was PlantVillage, containing 54,306 leaf images in JPEG format downloaded from GitHub, with data outside two families discarded during pre-processing. Three deep learning models were tested: transfer learning with InceptionV3 architecture and two custom CNNs (DFE and LCNN). LCNN model showed the best performance with training, validation, and testing accuracies of 99%, 99%, and 95%, respectively. In contrast, InceptionV3 achieved 96% training, 98% validation, and 92% testing accuracy, while DFE with 86% training, 94% validation, and 82% testing accuracy. Confusion matrix analysis showed difficulty distinguishing between healthy potatoes and potatoes with late blight, as well as cedar apple rust. These results highlights importance of developing specific model architectures rather than complex models for accurate multi-crop disease detection.

Abstrak

Dengan proyeksi populasi global sebesar 9,7 miliar pada tahun 2050, pertanian menghadapi tantangan signifikan dalam memastikan ketahanan pangan. Salah satu kendala utama adalah penyakit tanaman yang menurunkan hasil panen hingga 40% per tahun. Penelitian sebelumnya sering terbatas pada deteksi penyakit pada satu spesies tanaman, sehingga kurang mencerminkan kebutuhan multi-spesies dalam praktik pertanian nyata. Penelitian ini bertujuan untuk mengembangkan dan mengevaluasi sistem deteksi penyakit tanaman berbasis *deep learning* menggunakan *Convolutional Neural Networks* (CNN) yang diterapkan pada dua famili tanaman, *Solanaceae* dan *Rosaceae*. Dataset yang digunakan adalah PlantVillage, berisi 54.306 citra daun dalam format JPEG yang diunduh dari GitHub, dengan data di luar dua famili tersebut dibuang selama pra-pemrosesan. Tiga model *deep learning* diuji: *transfer learning* dengan arsitektur *InceptionV3* dan dua CNN kustom (DFE dan LCNN). Model LCNN menunjukkan kinerja terbaik dengan akurasi pelatihan, validasi, dan pengujian masing-masing sebesar 99%, 99%, dan 95%. Sebaliknya, *InceptionV3* mencapai akurasi pelatihan 96%, validasi 98%, dan pengujian 92%, sedangkan DFE dengan 86% pelatihan, 94% validasi, dan 82% akurasi pengujian. Analisis confusion matrix menunjukkan kesulitan membedakan antara kentang sehat dan kentang dengan *late blight*, serta *cedar apple rust*. Hasil ini menyoroti pentingnya pengembangan arsitektur model spesifik dibandingkan model yang kompleks untuk deteksi penyakit *multi-crop* secara akurat.

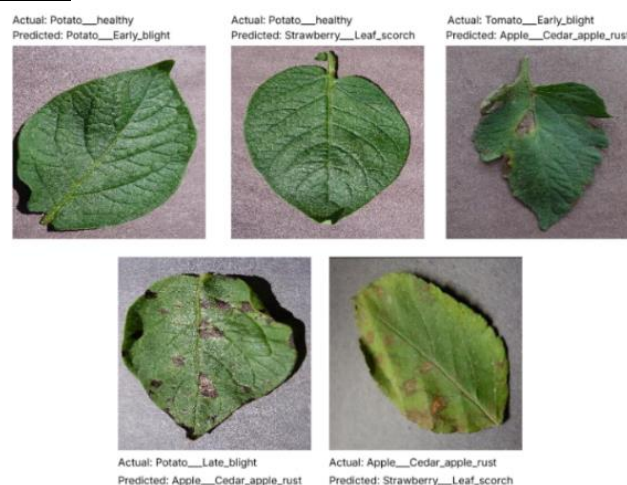
Poin Revisi No.3

Bab: Hasil dan Pembahasan

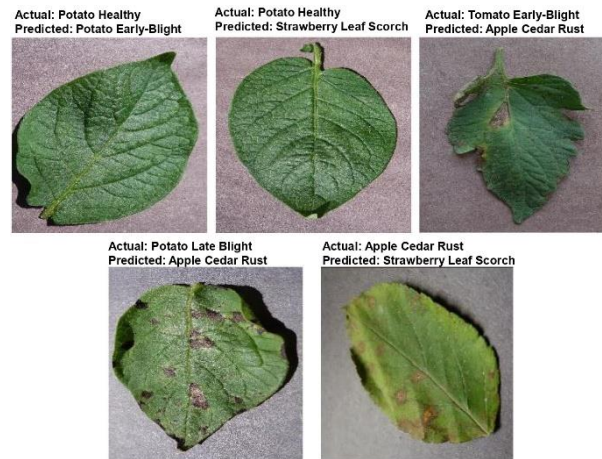
Komentar reviewer:

“masih ada gambar yang tulisan nya belum begitu jelas, jadi tidak tau apa maknanya, serta sulit menyesuaikan dengan keterangan gambar”

Sebelum Perbaikan:



Sesudah Perbaikan:



Poin Revisi No.4

Bab: Abstrak & Pendahuluan & Metode Penelitian

Komentar reviewer:

“masih bingung ini apakah membahas perbandingan kinerja dari CNN atau hanya 1 metode saja karena ada beberapa metode dalam CNN yang dibahas, lebih baik fokus saja, jangan terlalu melebar dan disesuaikan dengan judul atau memang mau membandingkan”

Sebelum Perbaikan:

- Abstrak:

This research aims to develop a deep learning-based plant disease detection system using Convolutional Neural Networks (CNN)

Penelitian ini bertujuan untuk mengembangkan sistem deteksi penyakit tanaman berbasis *deep learning* menggunakan *Convolutional Neural Networks (CNN)*

- Bab 1:

... Berbeda dengan penelitian sebelumnya yang umumnya terbatas pada satu spesies tanaman [8], [9], [10], [11], [12], [13], penelitian ini bertujuan mengembangkan model yang lebih adaptif dengan membandingkan kinerja *Transfer Learning* menggunakan *InceptionV3* dan CNN yang dirancang khusus. Pendekatan ini diharapkan dapat menghasilkan solusi yang lebih serbaguna dan mudah diterapkan dalam praktik pertanian di dunia nyata, sekaligus mengatasi keterbatasan generalisasi yang ditemukan pada penelitian-penelitian sebelumnya.

Sesudah Perbaikan:

- Abstrak:

This research aims to develop and evaluate deep learning-based plant disease detection system using Convolutional Neural Networks (CNN)

Penelitian ini bertujuan untuk mengembangkan dan mengevaluasi sistem deteksi penyakit tanaman berbasis *deep learning* menggunakan *Convolutional Neural Networks* (CNN)

- Bab 1:

...

Kontribusi utama penelitian ini adalah pengembangan dan mengevaluasi arsitektur *deep learning* untuk deteksi penyakit pada tanaman dari keluarga *Solanaceae* dan *Rosaceae*, dengan membandingkan dua pendekatan utama: *Transfer Learning* menggunakan *InceptionV3* dan CNN yang dirancang khusus. Berbeda dari penelitian sebelumnya yang umumnya terbatas pada deteksi penyakit untuk satu jenis tanaman saja [8], [9], [10], [11], [12], [13], melalui pendekatan multi-tanaman, penelitian ini tidak hanya menawarkan solusi praktis bagi petani yang umumnya menanam beragam jenis tanaman secara bersamaan, tetapi juga memberikan wawasan mendalam tentang keunggulan relatif dari setiap arsitektur melalui evaluasi dan perbandingan kedua pendekatan tersebut. Hasil evaluasi diharapkan memberikan solusi yang lebih serbaguna untuk praktik pertanian dunia nyata, sekaligus mengatasi keterbatasan generalisasi dari penelitian-penelitian sebelumnya.

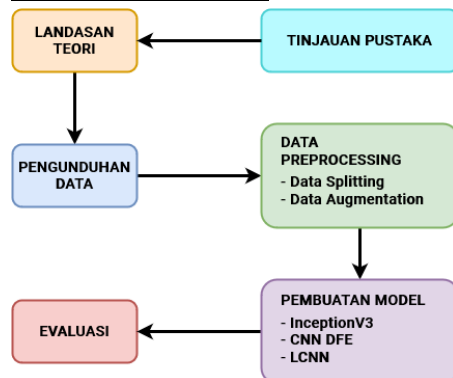
Hasil evaluasi diharapkan memberikan solusi yang lebih serbaguna untuk praktik pertanian dunia nyata, sekaligus mengatasi keterbatasan generalisasi dari penelitian-penelitian sebelumnya.

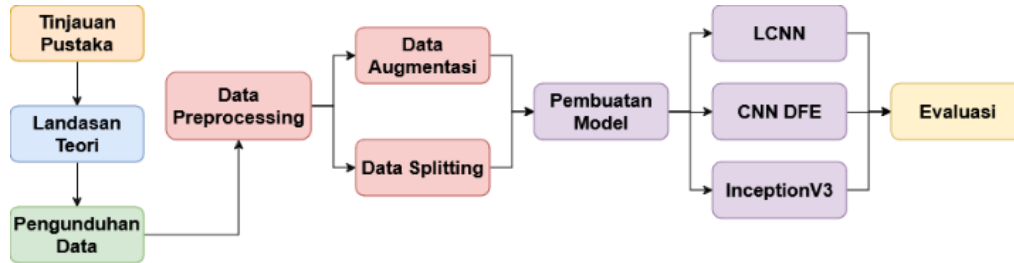
Poin Revisi No.5 Bab: Metode Penelitian

Komentar reviewer:

“lebih baik gambar 1 diganti dengan sebuah flowchart karena lebih baik, agar dapat dimengerti proses atau tahapan penelitian nya”

Sebelum Perbaikan:



Sesudah Perbaikan:

...

Poin Revisi No.6 Bab: Metode PenelitianKomentar reviewer:

“pada tabel 1 kenapa kok ada yang tulisan warna merah, apa maksud nya?”

Sebelum Perbaikan:

... Untuk penelitian ini, dataset telah dimodifikasi seperti yang terlihat pada Table 1 berikut, dengan menghapus data tanaman yang di luar spesies solanaceae dan rosaceae untuk memfasilitasi proses penelitian. Tanaman-tanaman yang dihapus dari dataset ditandai dengan teks yang berwarna merah dalam tabel. Ini termasuk spesies seperti Bluberi, Rasberi, Kedelai, Labu, Jagung, Anggur, dan Jeruk, yang tidak termasuk dalam dua spesies yang menjadi fokus penelitian ini. Penghapusan data tersebut dilakukan secara otomatis dalam python notebook menggunakan fitur Linux command untuk memastikan efisiensi dan akurasi.

<i>Dataset PlantVillage</i>	
<i>Apple Apple scab</i>	<i>Pepper bell healthy</i>
<i>Apple Black rot</i>	<i>Potato Early blight</i>
<i>Apple Cedar apple rust</i>	<i>Potato healthy</i>
<i>Apple healthy</i>	<i>Potato Late blight</i>
<i>Blueberry healthy</i>	<i>Raspberry healthy</i>
<i>Cherry (including sour) healthy</i>	<i>Soybean healthy</i>
<i>Cherry (including sour) Powdery mildew</i>	<i>Squash Powdery mildew</i>
<i>Corn (maize) Cercospora leaf spot</i>	<i>Strawberry healthy</i>
<i>Gray leaf spot</i>	<i>Strawberry Leaf scorch</i>
<i>Corn (maize) Common rust</i>	<i>Tomato Bacterial spot</i>
<i>Corn (maize) healthy</i>	<i>Tomato Early blight</i>
<i>Corn (maize) Northern Leaf Blight</i>	<i>Tomato healthy</i>
<i>Grape Black rot</i>	<i>Tomato Late blight</i>
<i>Grape Esca (Black Measles)</i>	<i>Tomato Leaf Mold</i>
<i>Grape healthy</i>	<i>Tomato Septoria leaf spot</i>
<i>Grape Leaf blight (Isariopsis Leaf Spot)</i>	

<i>Orange Haunglongbing (Citrus greening)</i>	<i>Tomato Spider mites Two-spotted spider mite</i>
<i>Peach Bacterial spot</i>	<i>Tomato Target Spot</i>
<i>Peach healthy</i>	<i>Tomato Tomato mosaic virus</i>
<i>Pepper bell Bacterial spot</i>	<i>Tomato Tomato Yellow Leaf Curl Virus</i>

Sesudah Perbaikan:

Dalam penelitian ini, dataset dimodifikasi sebagaimana ditunjukkan pada Tabel 1 dengan menghilangkan data tanaman di luar spesies Solanaceae dan Rosaceae untuk memudahkan proses penelitian. Modifikasi ini menghasilkan total 25 kelas. Tanaman yang dihilangkan dari dataset meliputi Bluberi, Rasberi, Kedelai, Labu, Jagung, Anggur, dan Jeruk karena tidak termasuk dalam kedua spesies yang menjadi fokus penelitian. Selain itu, setiap kelas diberi kode untuk meningkatkan keterbacaan.

<i>Dataset PlantVillage</i>	
<i>Apple Apple scab (A-SCAB)</i>	<i>Strawberry healthy (S-HLTH)</i>
<i>Apple Black rot (A-ROT)</i>	<i>Strawberry Leaf scorch (S-LSCR)</i>
<i>Apple Cedar apple rust (A-CRUST)</i>	<i>Tomato Bacterial spot (T-BSPT)</i>
<i>Apple healthy (A-HLTH)</i>	<i>Tomato Early blight (T-EBLT)</i>
<i>Cherry healthy (C-HLTH)</i>	<i>Tomato healthy (T-HLTH)</i>
<i>Cherry Powdery mildew (C-PMIL)</i>	<i>Tomato Late blight (T-LBLT)</i>
<i>Peach Bacterial spot (PE-BSPT)</i>	<i>Tomato Leaf Mold (T-LM)</i>
<i>Peach healthy (PE-HLTH)</i>	<i>Tomato Septoria leaf spot (T-SSP)</i>
<i>Pepper bell Bacterial spot (BP-BSPT)</i>	<i>Tomato Spider mites Two-spotted spider mite (T-SM)</i>
<i>Pepper bell healthy (BP-HLTH)</i>	<i>Tomato Target Spot (T-TS)</i>
<i>Potato Early blight (P-EBLT)</i>	<i>Tomato Tomato mosaic virus (T-MV)</i>
<i>Potato healthy (P-HLTH)</i>	<i>Tomato Tomato Yellow Leaf Curl Virus (T-YCV)</i>
<i>Potato Late blight (P-LBLT)</i>	

Poin Revisi No.7 Bab: Hasil dan Pembahasan

Komentar reviewer:

“penelitian ini menekankan akurasi atau identifikasi ? karena masih belum jelas”

Sebelum Perbaikan:

Meskipun ketiga model menunjukkan kinerja yang menjanjikan, analisis lebih lanjut mengungkapkan beberapa keterbatasan yang perlu diperhatikan. Salah satu keterbatasan utama adalah terjadinya hasil positif palsu, seperti yang ditunjukkan pada Gambar 11. Keadaan ini menunjukkan bahwa model tersebut masih menghadapi tantangan dalam membedakan antara beberapa penyakit tanaman dan kondisi daun yang sehat dengan akurasi yang tinggi.

...

Citra 3, 4, dan 5 kemungkinan menghasilkan false positive karena kemiripan gejala antar penyakit yang diidentifikasi, serta kesamaan bentuk dan warna daun dari jenis-jenis tanaman tersebut. Sebagai contoh, pola kerusakan daun pada penyakit apple cedar rust dan tomato early blight cenderung serupa, sehingga sulit dibedakan oleh model. Ditambah lagi, tekstur daun tanaman seperti apel dan kentang memiliki kemiripan yang dapat berkontribusi pada kesalahan identifikasi.

Sementara itu, penyebab kesalahan pada citra 1 dan 2, di mana daun kentang sehat teridentifikasi sebagai penyakit potato early blight atau strawberry leaf scorch, tampak kurang jelas. Meskipun daun sehat seharusnya berwarna hijau, ada kemungkinan model terkecoh oleh variasi kecil pada warna daun, seperti perubahan intensitas hijau atau kondisi pencahayaan yang tidak konsisten. Variasi-variasi ini mungkin dianggap oleh model sebagai gejala penyakit, padahal sebenarnya tidak ada tanda-tanda penyakit yang nyata.

Sesudah Perbaikan:

Meskipun ketiga model menunjukkan kinerja yang menjanjikan dengan akurasi InceptionV3 mencapai 0.92, LCNN 0.95, dan CNN DFE 0.82, analisis lebih lanjut mengungkapkan beberapa keterbatasan yang perlu diperhatikan. Salah satu keterbatasan utama adalah terjadinya hasil positif palsu, seperti yang ditunjukkan pada Gambar 11. Keadaan ini menunjukkan bahwa model tersebut masih menghadapi tantangan dalam membedakan antara beberapa penyakit tanaman dan kondisi daun yang sehat dengan akurasi yang tinggi.

...

Citra 3, 4, dan 5 kemungkinan menghasilkan false positive karena kemiripan gejala antar penyakit yang diidentifikasi, serta kesamaan bentuk dan warna daun dari jenis-jenis tanaman tersebut. Sebagai contoh, pola kerusakan daun pada penyakit apple cedar rust yang memiliki presisi 1.00 tetapi recall hanya 0.66-0.68 pada model InceptionV3 dan LCNN, dan tomato early blight dengan presisi 1.00 tetapi recall 0.62-0.67 pada model InceptionV3 dan CNN DFE cenderung serupa, sehingga sulit dibedakan oleh model.

Sementara itu, penyebab kesalahan pada citra 1 dan 2, di mana daun kentang sehat teridentifikasi sebagai penyakit potato early blight (yang memiliki recall

0.98-1.00 dan presisi 0.87-0.90 pada ketiga model) atau strawberry leaf scorch (dengan recall 0.89-1.00 dan presisi 0.78-0.85 pada ketiga model), tampak kurang jelas. Data ini menunjukkan bahwa model cenderung over-confident dalam mengidentifikasi kedua penyakit tersebut (recall tinggi), yang bisa menjelaskan kenapa daun sehat bisa salah teridentifikasi sebagai penyakit ini.

Poin Revisi No.8	Bab: Hasil dan Pembahasan
-------------------------	----------------------------------

Komentar reviewer:

“hasil confusion matrixnya belum jelas”

Sebelum Perbaikan:

Hasil dari *confusion matrix* untuk model *InceptionV3* dapat dilihat pada Gambar 8.

Sesudah Perbaikan:

Hasil dari *confusion matrix* untuk model *InceptionV3* dapat dilihat pada Gambar 8. Dalam confusion matrix berikut, setiap kelas penyakit direpresentasikan dengan singkatan yang sesuai dengan daftar kelas pada Tabel 1, seperti '*A-SCAB*' untuk *Apple Scab*.

BAB II

PENAMBAHAN FITUR SISTEM

2.1 Target Penambahan Fitur Sistem

Penambahan fitur sistem dalam Proyek Profesional (PP) ini merupakan kelanjutan dari Proyek Utama Informatika (PUI). Pada PUI, peneliti mengikuti konversi mata kuliah Google Bangkit di jalur *Machine Learning*. Dalam proyek tim capstone Google Bangkit, peneliti bertanggung jawab atas implementasi model *Machine Learning* ke dalam aplikasi Android menggunakan FastAPI. Aplikasi yang dihasilkan adalah prototipe sederhana, yang kemudian dikembangkan lebih lanjut oleh anggota tim dari jalur Android Development.

Namun, karena kontribusi peneliti dalam proyek capstone terbatas pada integrasi FastAPI dan model *Machine Learning*, pada PUI peneliti memutuskan untuk melanjutkan dan mengembangkan prototipe tersebut secara mandiri. Fokus utama peneliti adalah memperluas aplikasi tersebut serta mengembangkan sistem *Machine Learning* lebih lanjut.

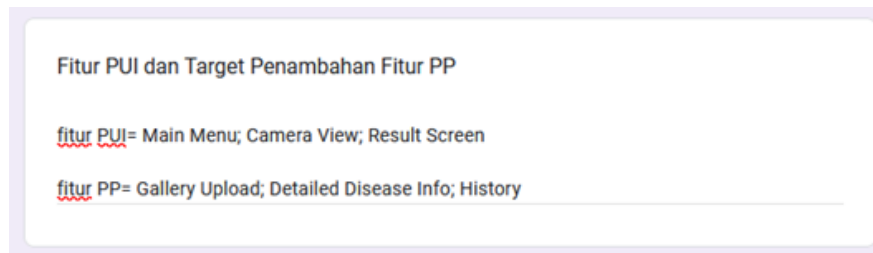
Saat Seminar PUI, peneliti mengusulkan beberapa penambahan fitur yang akan diimplementasikan dalam Proyek Profesional ini. Usulan tersebut dapat dilihat pada Gambar 2.1 di bawah ini.



Gambar 2.1 Usulan Penambahan Fitur pada Poster Seminar PUI

Selanjutnya, pada pertemuan pertama PP, peneliti mengisi formulir 'Form Isian untuk Mahasiswa Proyek Profesional, Penulisan Karya Ilmiah, dan Kerja

Praktik'. Formulir ini berisi rincian fitur yang telah dikembangkan pada PUI serta fitur-fitur tambahan yang akan diimplementasikan pada Proyek Profesional. Salah satu bagian dari isian formulir tersebut ditunjukkan pada Gambar 2.2 berikut ini sebagai bukti dokumentasi perencanaan fitur.



Fitur PUI dan Target Penambahan Fitur PP

fitur PUI= Main Menu; Camera View; Result Screen

fitur PP= Gallery Upload; Detailed Disease Info; History

Gambar 2.2 Target Penambahan Fitur Sistem dari Google Form

2.2 Perbandingan Fitur Sistem

Selama pelaksanaan Proyek Profesional, prototipe yang dikembangkan pada PUI telah diperbarui secara signifikan. Penyempurnaan utama difokuskan pada penambahan fitur-fitur baru, sehingga sistem menjadi lebih komprehensif dan siap untuk digunakan. Tabel 2.1 di bawah ini menggambarkan perbandingan antara fitur sistem pada prototipe awal dan fitur tambahan yang dikembangkan selama Proyek Profesional.

Tabel 2.1 Perbandingan Fitur Sistem

Fitur Sistem pada Prototipe PUI	Tambahan Fitur Sistem pada Proyek Profesional
1. <i>Main Menu</i> 2. <i>Camera View</i> 3. <i>Result Screen</i> 4. Dataset terbatas	1. <i>Gallery Upload</i> 2. <i>Detailed Disease Information</i> 3. <i>History</i> 4. Penambahan Data 5. UI yang lebih responsif, <i>user-friendly</i>, dan <i>robust</i>

BAB III

Pengenalan Antar Muka Sistem

3.1 Halaman *Splash Screen*

Halaman *Splash Screen* merupakan halaman pertama yang muncul saat sistem mulai berjalan. Halaman ini berfungsi sebagai pengenalan awal aplikasi kepada pengguna, memberikan waktu bagi sistem untuk melakukan proses inisialisasi, serta menciptakan kesan profesional pada aplikasi. Tampilan halaman ini hanya terdiri dari satu gambar yang terletak di tengah, yang dapat berupa logo atau elemen visual lain yang mewakili identitas aplikasi. Tampilan Halaman *Splash Screen* dapat dilihat pada Gambar 3.1.



Gambar 3.1 Tampilan Halaman *Splash Screen*

Berikut ini adalah potongan kode program yang digunakan untuk membuat halaman *splash screen* seperti yang ditampilkan pada Gambar 3.1, yang dapat dilihat pada Gambar 3.2 di bawah ini.



```

package com.allanbil214.leafyapp

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.app.AppCompatActivityDelegate
import androidx.core.splashscreen.SplashScreen.Companion.installSplashScreen

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        // Enable splash screen
        installSplashScreen()

        super.onCreate(savedInstanceState)
        AppCompatActivityDelegate.setDefaultNightMode(AppCompatActivityDelegate.MODE_NIGHT_NO)
        setContentView(R.layout.activity_main)}}

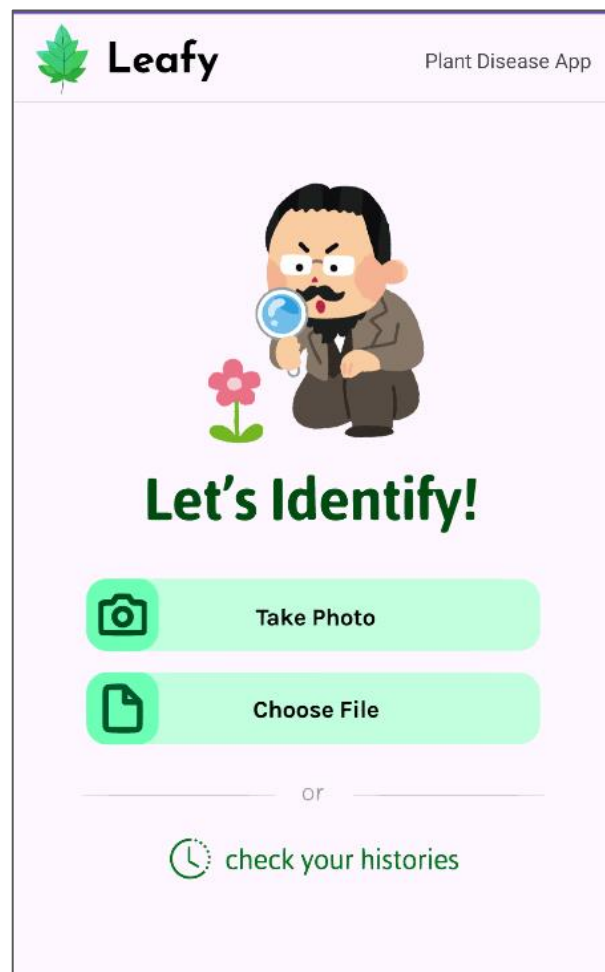
```

Gambar 3.2 Kode Program Halaman *Splash Screen*

3.2 Halaman Utama

Halaman utama adalah halaman pertama yang tampil setelah aplikasi dimulai dan berfungsi sebagai antarmuka utama bagi pengguna untuk berinteraksi dengan sistem. Pada bagian atas halaman, terdapat judul aplikasi "*Leafy*" dan deskripsi singkat "*Plant Disease App*" yang menjelaskan fungsi aplikasi. Di bagian tengah, terdapat ilustrasi karakter dengan kaca pembesar yang merepresentasikan proses identifikasi penyakit tanaman.

Di bawah ilustrasi tersebut, terdapat dua tombol interaktif, yaitu "*Take Photo*" dan "*Choose File*", yang memungkinkan pengguna mengunggah gambar daun untuk dianalisis. Fitur tambahan seperti "*Check Your Histories*" untuk melihat riwayat identifikasi. Tampilan Halaman Utama dapat dilihat pada Gambar 3.3.



Gambar 3.3 Tampilan Halaman Utama

Selanjutnya, kode program yang digunakan untuk membuat halaman utama, seperti yang ditunjukkan pada Gambar 3.3, dapat dilihat pada Gambar 3.4 sampai Gambar 3.9 berikut.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <ImageView
        android:id="@+id/button_takephoto"
        android:layout_width="394dp"
        android:layout_height="56dp"
        android:layout_marginTop="24dp"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.47"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/imageView5"
        app:srcCompat="@drawable/takephoto_button" />

    <ImageView
        android:id="@+id/button_choosefile"
        android:layout_width="394dp"
        android:layout_height="56dp"
        android:layout_marginTop="8dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.47"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/button_takephoto"
        app:layout_constraintVertical_bias="0.0"
        app:srcCompat="@drawable/choosefile_button" />

    <ImageView
        android:id="@+id/historyImageView"
        android:layout_width="201dp"
        android:layout_height="31dp"
        android:layout_marginTop="16dp"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/imageView7"
        app:srcCompat="@drawable/history_button" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

Gambar 3.4 Kode Program untuk Tampilan Halaman Utama

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    historyButton.setOnClickListener {
        val intent = Intent(this, HistoryActivity::class.java)
        startActivity(intent)
    }
}

```

Gambar 3.5 Kode Program Tombol Menuju Halaman Riwayat


```

private fun processImage(bitmap: Bitmap) {
    progressDialog.show() // Show the loading dialog
    val resizedBitmap = resizeBitmap(bitmap, 640, 420)
    selectedImage.setImageBitmap(resizedBitmap)
    imageBase64 = encodeImage(resizedBitmap)
    sendImage(imageBase64!!)}

private fun resizeBitmap(bitmap: Bitmap, maxWidth: Int, maxHeight: Int): Bitmap {
    val originalWidth = bitmap.width
    val originalHeight = bitmap.height
    val scaleFactor = min(maxWidth.toFloat() / originalWidth, maxHeight.toFloat() / originalHeight)
    val newWidth = (originalWidth * scaleFactor).toInt()
    val newHeight = (originalHeight * scaleFactor).toInt()

    return Bitmap.createScaledBitmap(bitmap, newWidth, newHeight, true)}

private fun encodeImage(bitmap: Bitmap): String {
    val byteArrayOutputStream = ByteArrayOutputStream()
    bitmap.compress(Bitmap.CompressFormat.JPEG, 90, byteArrayOutputStream) // Reduced quality to 90%
    val byteArray = byteArrayOutputStream.toByteArray()
    return Base64.encodeToString(byteArray, Base64.DEFAULT)}

```

Gambar 3.6 Kode Program Pemrosesan Gambar

```

private fun sendImage(base64String: String) {
    val retrofit = Retrofit.Builder()
        .baseUrl("https://hyena-pure-violently.ngrok-free.app/")
        .addConverterFactory(GsonConverterFactory.create())
        .build()

    val apiService = retrofit.create(ApiService::class.java)
    val request = PredictRequest(base64_encoded = base64String)
    val call = apiService.predictDisease(request)

    call.enqueue(object : Callback<PredictionResult> {
        override fun onResponse(call: Call<PredictionResult>, response: Response<PredictionResult>) {
            progressDialog.dismiss()

            if (response.isSuccessful) {
                val result = response.body()?.predicted_class
                val plant = response.body()?.plant_name
                val disease = response.body()?.plant_disease
                val url = response.body()?.plant_url
                startResultActivity(result, plant, disease, url)
            } else {
                val result = "API Error: ${response.code()}"
                val plant = "API Error: ${response.code()}"
                val disease = "API Error: ${response.code()}"
                val url = "API Error: ${response.code()}"
                startResultActivity(result, plant, disease, url)
            }
        }

        override fun onFailure(call: Call<PredictionResult>, t: Throwable) {
            progressDialog.dismiss() // Dismiss the dialog in case of failure
            val result = "Network Error: ${t.message}"
            val plant = "Network Error: ${t.message}"
            val disease = "Network Error: ${t.message}"
            val url = "Network Error: ${t.message}"
            startResultActivity(result, plant, disease, url)
        }
    })
}

```

Gambar 3.7 Kode Program Pengiriman Gambar

```

interface ApiService {
    @POST("predict")
    fun predictDisease(@Body request: PredictRequest): Call<PredictionResult>

    @GET("disease-info/{disease}")
    fun getDiseaseInfo(@Path("disease") disease: String): Call<DiseaseInfo>}

data class PredictRequest(val base64_encoded: String)

data class PredictionResult(
    val predicted_class: String,
    val plant_name: String,
    val plant_disease: String,
    val plant_url: String)

data class DiseaseInfo(val disease_info: String)

```

Gambar 3.8 Kode Program *API Service*

```

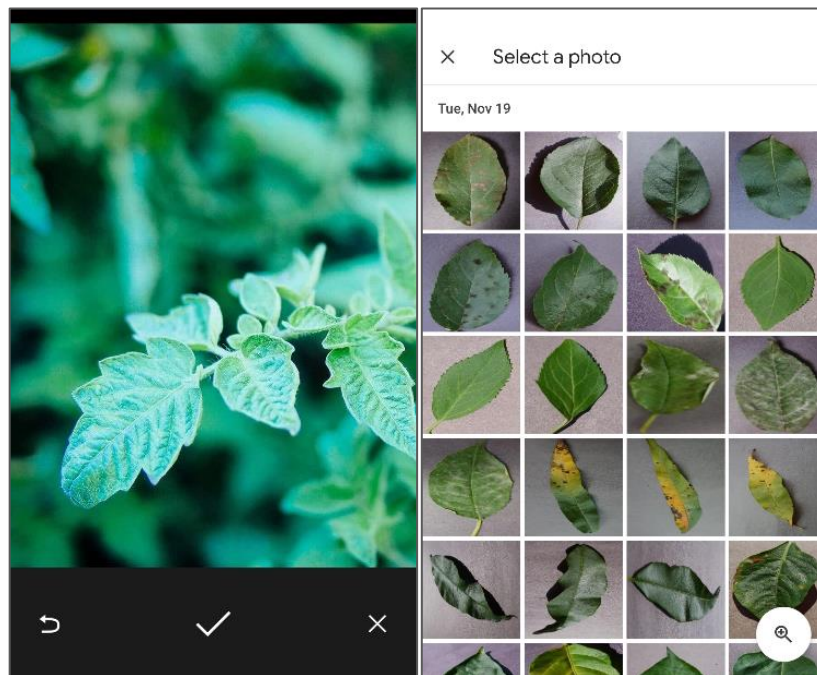
private fun startResultActivity(result: String?, plant: String?, disease: String?, url: String?) {
    val intent = Intent(this, ResultActivity::class.java)
    intent.putExtra("RESULT", result)
    intent.putExtra("PLANT", plant)
    intent.putExtra("DISEASE", disease)
    intent.putExtra("URL", url)
    intent.putExtra("IMAGE", imageBase64)
    startActivity(intent)
}

```

Gambar 3.9 Kode Program Tombol Menuju Halaman Hasil

3.3 Halaman Pengambilan Gambar

Pada halaman pengambilan gambar, aplikasi ini memanfaatkan fitur kamera dan galeri bawaan yang disediakan oleh sistem operasi perangkat, sesuai dengan vendor masing-masing. Pengguna dapat memilih untuk langsung mengambil gambar daun menggunakan kamera. Sebagai alternatif, pengguna dapat mengakses galeri untuk memilih gambar daun yang sudah ada, sehingga mempermudah proses input gambar tanpa perlu mengambil gambar baru. Contoh tampilan kamera dan galeri dapat dilihat pada Gambar 3.10 di bawah.



Gambar 3.10 Tampilan Halaman Kamera dan Galeri

Implementasi fitur kamera dan galeri pada halaman ini ditunjukkan dalam potongan kode pada Gambar 3.11.

```

private fun openGallery() {
    val intent = Intent(Intent.ACTION_PICK)
    intent.type = "image/*"
    startActivityForResult(intent, REQUEST_IMAGE_PICK)}

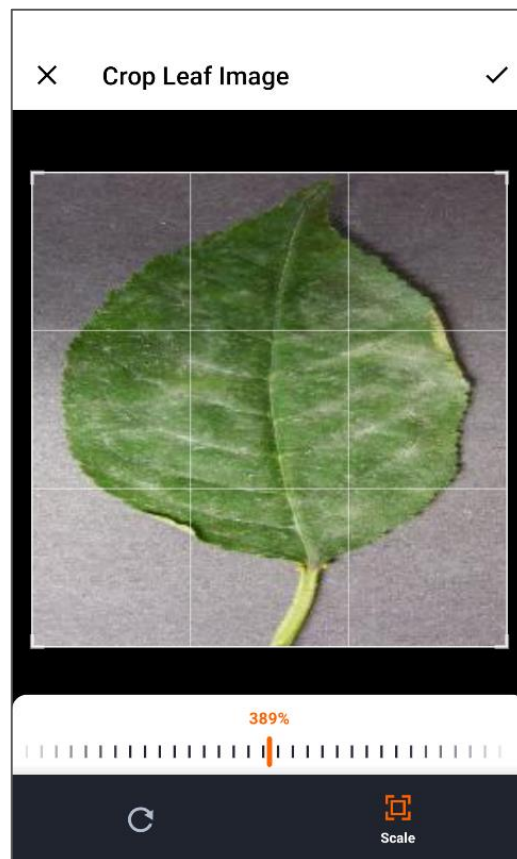
private fun openCamera() {
    Intent(MediaStore.ACTION_IMAGE_CAPTURE).also { takePictureIntent ->
        takePictureIntent.resolveActivity(packageManager)?.also {
            val photoFile: File? = try {
                createImageFile()
            } catch (ex: IOException) {
                null
            }
            photoFile?.also {
                val photoURI: Uri = FileProvider.getUriForFile(
                    this,
                    "com.allanbil214.leafyapp.fileprovider",
                    it
                )
                takePictureIntent.putExtra(MediaStore.EXTRA_OUTPUT, photoURI)
                startActivityForResult(takePictureIntent, REQUEST_IMAGE_CAPTURE)
            }
        }
    }
}

```

Gambar 3.11 Kode Program untuk Membuka Kamera dan Galeri

3.4 Halaman *Cropping* Gambar

Pada halaman *cropping* gambar, pengguna diberikan kemudahan untuk memotong gambar sesuai kebutuhan. Pengguna dapat menyesuaikan batas *crop* pada gambar yang ditampilkan, sehingga dapat memilih area gambar yang diinginkan dengan presisi. Aplikasi ini menggunakan *library uCrop* yang dikembangkan oleh Yalantis di GitHub, yang memungkinkan integrasi fungsionalitas *cropping* yang efisien dan responsif pada berbagai perangkat. Selain itu, *uCrop* mendukung berbagai fitur tambahan seperti pengaturan *aspect ratio*, rotasi, dan *zoom*, yang meningkatkan fleksibilitas dan kenyamanan pengguna. Tampilan fitur *cropping* gambar dapat dilihat pada Gambar 3.12 berikut.



Gambar 3.12 Tampilan Halaman *Cropping* Gambar

Potongan kode program untuk fungsi *cropping* gambar dapat dilihat pada Gambar 3.13.

```

private fun startCrop(sourceUri: Uri) {
    val destinationUri = Uri.fromFile(File(cacheDir, "cropped_${System.currentTimeMillis()}.jpg"))

    val uCrop = UCrop.of(sourceUri, destinationUri)
        .withAspectRatio(1f, 1f) // You can adjust this ratio as needed
        .withMaxResultSize(640, 420) // Match your existing resize dimensions

    // Customize the cropping window
    val options = UCrop.Options().apply {
        setCircleDimmedLayer(false) // Set to true for circular crop
        setShowCropFrame(true)
        setShowCropGrid(true)
        setToolbarColor(ContextCompat.getColor(this@MainActivity, R.color.white))
        setStatusBarColor(ContextCompat.getColor(this@MainActivity, R.color.white))
        setToolbarTitle("Crop Leaf Image")
        setFreeStyleCropEnabled(true) // Allow free-form cropping
    }

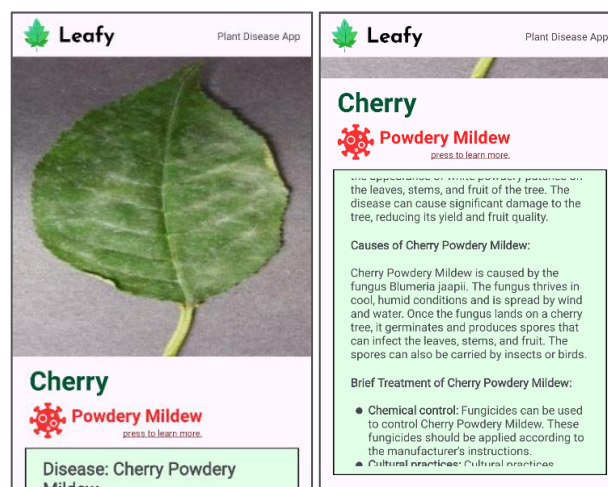
    uCrop.withOptions(options)
    uCrop.start(this)
}

```

Gambar 3.13 Kode Program untuk Fungsi *Cropping* Gambar

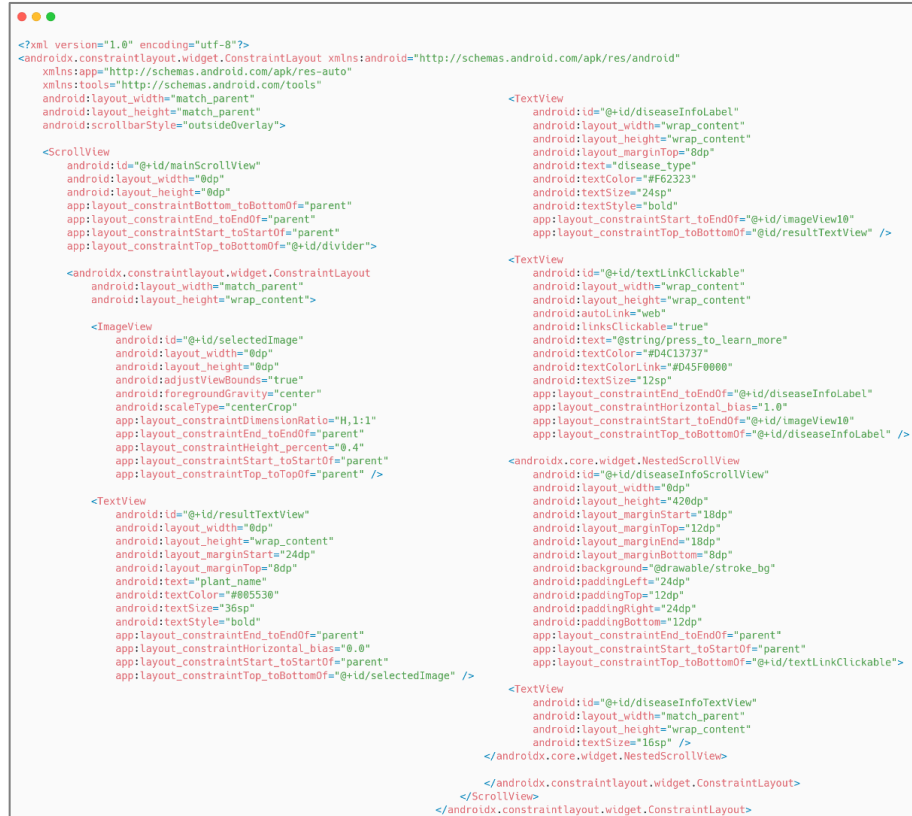
3.5 Halaman Hasil

Pada halaman hasil, gambar yang diambil ditampilkan di bagian atas halaman. Di bawah gambar tersebut, ditampilkan jenis tanaman dan jenis penyakit, disertai dengan tombol untuk membuka tautan penjelasan penyakit yang lebih detail. Selain itu, terdapat juga penjelasan singkat mengenai penyakit tersebut di bawahnya. Hasil identifikasi juga akan disimpan sebagai riwayat dalam bentuk file *JSON* di penyimpanan lokal. Tampilan halaman hasil dapat dilihat pada Gambar 3.14 berikut.

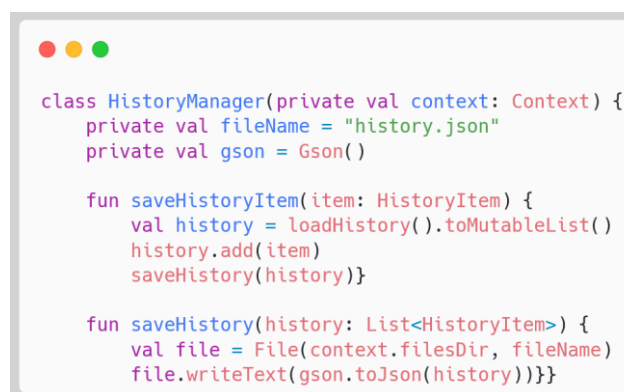


Gambar 3.14 Tampilan Halaman Hasil Identifikasi

Kode yang digunakan untuk menampilkan Halaman Hasil, termasuk informasi tanaman dan penyakit, terdapat pada Gambar 3.15 hingga Gambar 3.20.



Gambar 3.15 Kode Program Tampilan Halaman Hasil



Gambar 3.16 Kode Program Penyimpanan Riwayat Identifikasi

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_result)

    try {
        selectedImage = findViewById(R.id.selectedImage)
        resultTextView = findViewById(R.id.resultTextView)
        textLinkClickable = findViewById(R.id.textLinkClickable)
        diseaseInfoLabel = findViewById(R.id.diseaseInfoLabel)
        diseaseInfoTextView = findViewById(R.id.diseaseInfoTextView)

        progressDialog = ProgressDialog(this).apply {
            setMessage("Fetching information, please wait...")
            setCancelable(false)}
        result = intent.getStringExtra("RESULT") ?: "Unknown Result"
        plant = intent.getStringExtra("PLANT") ?: "Unknown Plant"
        disease = intent.getStringExtra("DISEASE") ?: "Unknown Disease"
        url = intent.getStringExtra("URL")
        imageBase64 = intent.getStringExtra("IMAGE")

        resultTextView.text = plant
        diseaseInfoLabel.text = disease
        displayImage(imageBase64)
        result?.let { fetchDiseaseInfo(it) }
        setupUrlClickListener()
    } catch (e: Exception) {handleInitializationError(e)}}

```

Gambar 3.17 Kode Program Inisialisasi Halaman Hasil

```

private fun displayImage(imageBase64: String?) {
    try {
        imageBase64?.let {
            val imageBytes = Base64.decode(it, Base64.DEFAULT)
            val bitmap = BitmapFactory.decodeByteArray(imageBytes, 0, imageBytes.size)

            if (bitmap != null) {
                val aspectRatio = bitmap.width.toFloat() / bitmap.height.toFloat()
                val layoutParams = selectedImage.layoutParams as ConstraintLayout.LayoutParams
                layoutParams.dimensionRatio = "H,$aspectRatio"
                selectedImage.layoutParams = layoutParams
                selectedImage.setImageBitmap(bitmap)
            } else {
                throw IllegalStateException("Failed to decode bitmap")}
        } ?: run {
            selectedImage.setImageResource(android.R.drawable.ic_menu_camera)}
    } catch (e: Exception) {
        e.printStackTrace()
        Toast.makeText(this, "Error displaying image: ${e.localizedMessage}", Toast.LENGTH_SHORT).show()
        selectedImage.setImageResource(android.R.drawable.ic_menu_camera)
    }
}

```

Gambar 3.18 Kode Program untuk Menampilkan Gambar Hasil


```

private fun setupUrlClickListener() {
    url?.let { urlString ->
        textLinkClickable.apply {
            setOnClickListener {
                try {
                    val browserIntent = Intent(Intent.ACTION_VIEW, Uri.parse(urlString))
                    startActivity(browserIntent)
                } catch (e: Exception) {
                    Toast.makeText(
                        this@ResultActivity,
                        "Unable to open link: ${e.localizedMessage}",
                        Toast.LENGTH_SHORT
                    ).show()
                }
            }
        }
    }
}

} ?: run {
    textLinkClickable.setTextColor(
        ContextCompat.getColor(this, R.color.d45f0000))
    textLinkClickable.isClickable = false
}

```

Gambar 3.19 Kode Program untuk Membuat Tautan Teks

```

private fun fetchDiseaseInfo(disease: String) {
    progressDialog.show()
    try {
        val retrofit = Retrofit.Builder()
            .baseUrl("https://hyena-pure-violently.ngrok-free.app/")
            .addConverterFactory(GsonConverterFactory.create())
            .build()
        val apiService = retrofit.create(ApiService::class.java)
        val call = apiService.getDiseaseInfo(disease)

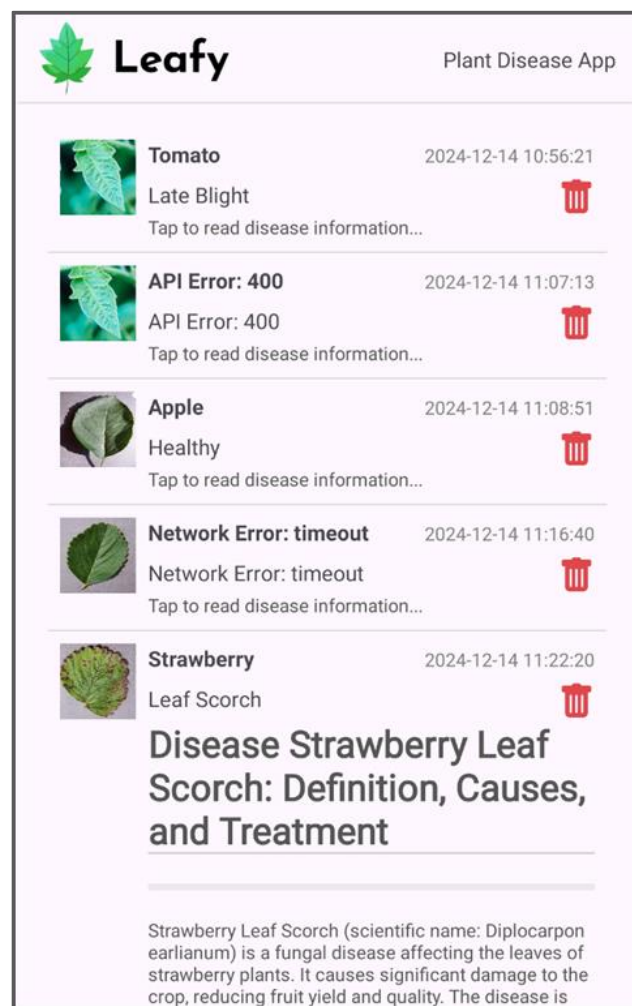
        call.enqueue(object : Callback<DiseaseInfo> {
            override fun onResponse(call: Call<DiseaseInfo>, response: Response<DiseaseInfo>) {
                progressDialog.dismiss()
                try {
                    if (response.isSuccessful) {
                        val info = response.body()?.disease_info ?: "No information available."
                        val markwon = Markwon.create(this@ResultActivity)
                        markwon.setMarkdown(diseaseInfoTextView, info)
                        saveToHistory(info, currentDate)
                    } else {
                        val errorInfo = "API Error: ${response.code()} - ${response.message()}"
                        askToSaveError(errorInfo, currentDate)
                        throw Exception(errorInfo)
                    }
                } catch (e: Exception) {
                    handleDiseaseInfoError(e)}
            override fun onFailure(call: Call<DiseaseInfo>, t: Throwable) {
                progressDialog.dismiss()
                val errorInfo = "Network Error: ${t.message}"
                askToSaveError(errorInfo, currentDate)
                handleDiseaseInfoError(t)}}
        } catch (e: Exception) {
            progressDialog.dismiss()
            val errorInfo = "Error: ${e.message}"
            askToSaveError(errorInfo, currentDate)
            handleDiseaseInfoError(e)}
    }
}

```

Gambar 3.20 Kode Program untuk Mengambil Informasi Penyakit

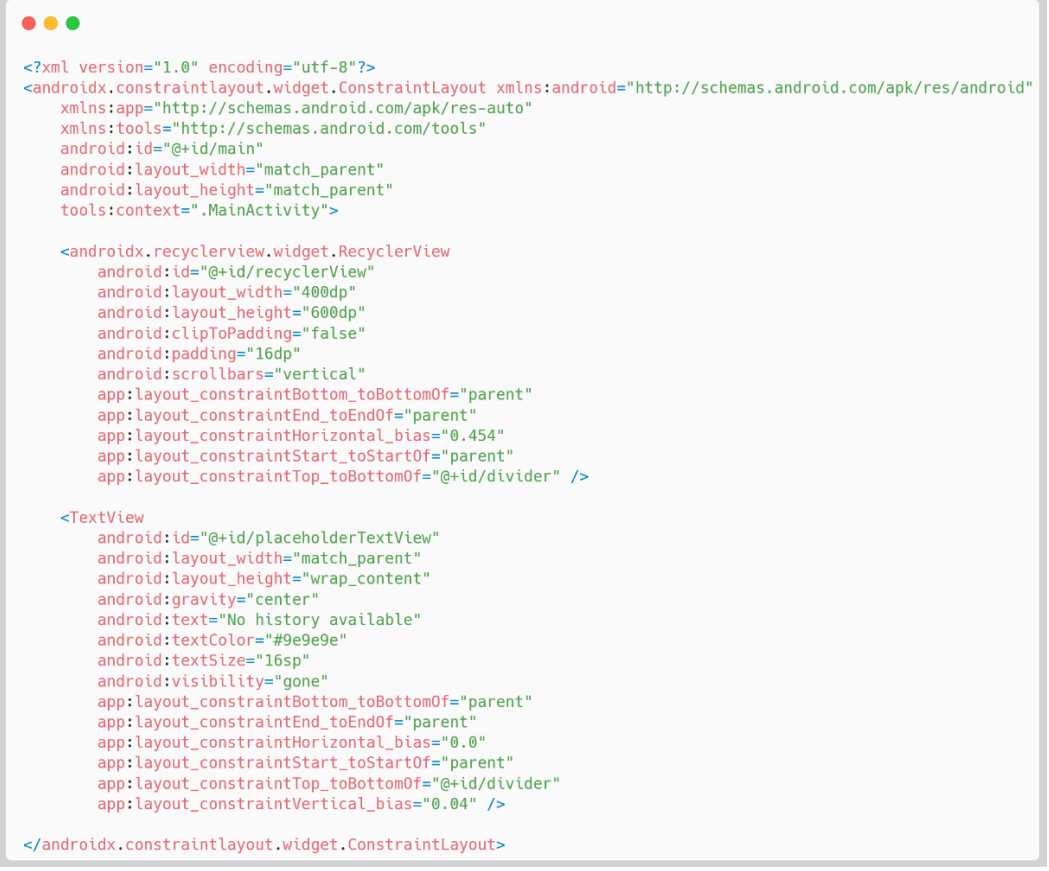
3.6 Halaman Riwayat

Halaman riwayat berfungsi untuk menampilkan hasil identifikasi yang telah dilakukan sebelumnya. Pengguna dapat melihat kembali riwayat hasil analisis dan menghapus entri yang tidak diinginkan. Tampilan halaman riwayat dapat dilihat pada Gambar 3.21.



Gambar 3.21 Tampilan Halaman Riwayat Identifikasi

Implementasi halaman riwayat beserta fungsinya dapat dilihat melalui kode pada Gambar 3.22 hingga Gambar 3.29.



```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerView"
        android:layout_width="400dp"
        android:layout_height="600dp"
        android:clipToPadding="false"
        android:padding="16dp"
        android:scrollbars="vertical"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.454"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/divider" />

    <TextView
        android:id="@+id/placeholderTextView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="No history available"
        android:textColor="#9e9e9e"
        android:textSize="16sp"
        android:visibility="gone"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.0"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/divider"
        app:layout_constraintVertical_bias="0.04" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

Gambar 3.22 Kode Program Tampilan Utama Halaman Riwayat

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:padding="8dp">

    <!-- Image Section -->
    <ImageView
        android:id="@+id/imageView"
        android:layout_width="50dp"
        android:layout_height="50dp"
        android:scaleType="centerCrop"
        android:layout_marginEnd="8dp" />

    <!-- Text Section -->
    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:orientation="vertical">

        <!-- Plant Name and Date -->
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:layout_marginBottom="4dp">

            <TextView
                android:id="@+id/plantTextView"
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:text="Plant Name"
                android:textStyle="bold"
                android:textSize="14sp" />

            <TextView
                android:id="@+id/dateTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Date here"
                android:textSize="12sp"
                android:textColor="#808080" />

        </LinearLayout>

        <!-- Disease Name with Delete Button -->
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:gravity="center_vertical">

            <TextView
                android:id="@+id/diseaseTextView"
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:text="Disease Name"
                android:textSize="14sp" />

            <ImageView
                android:id="@+id/deleteButton"
                android:layout_width="20dp"
                android:layout_height="20dp"
                android:src="@drawable/ic_trash" />

        </LinearLayout>

        <!-- Disease Info -->
        <TextView
            android:id="@+id/diseaseInfoTextView"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Disease Info..."
            android:ellipsize="end"
            android:maxLines="1"
            android:textSize="12sp"
            android:textColor="#555555" />

    </LinearLayout>

</LinearLayout>

```

Gambar 3.23 Kode Program Tampilan Item Riwayat

```

class HistoryManager(private val context: Context) {
    private val fileName = "history.json"
    private val gson = Gson()

    fun loadHistory(): List<HistoryItem> {
        val file = File(context.filesDir, fileName)
        if (!file.exists()) {
            return emptyList()
        }
        val json = file.readText()
        val type = object : TypeToken<List<HistoryItem>>() {}.type
        return gson.fromJson(json, type)}

```

Gambar 3.24 Kode Program untuk Memuat Item Riwayat



```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_history)
    historyManager = HistoryManager(this)
    recyclerView = findViewById(R.id.recyclerView)
    placeholderTextView = findViewById(R.id.placeholderTextView)

    setupRecyclerView()

    private fun setupRecyclerView() {
        val historyList = historyManager.loadHistory().toMutableList()
        recyclerView.layoutManager = LinearLayoutManager(this)
        adapter = HistoryAdapter(historyList) { historyItem, position ->
            showDeleteConfirmationDialog(historyItem, position)}

        recyclerView.adapter = adapter
        togglePlaceholderVisibility(historyList)}

```

Gambar 3.25 Kode Program Inisialisasi Halaman Riwayat



```

private fun deleteHistoryItem(historyItem: HistoryItem, position: Int) {
    try {
        adapter.removeItem(position)
        val updatedList = historyManager.loadHistory().toMutableList()
        updatedList.removeAll { it == historyItem }
        historyManager.saveHistory(updatedList)
        togglePlaceholderVisibility(updatedList)
    } catch (e: Exception) {e.printStackTrace()}}

```

Gambar 3.26 Kode Program untuk Menghapus Item Riwayat dari Database



```

private fun showDeleteConfirmationDialog(historyItem: HistoryItem, position: Int) {
    AlertDialog.Builder(this)
        .setTitle("Delete History")
        .setMessage("Are you sure you want to delete this history item?")
        .setPositiveButton("Delete") { _, _ ->
            deleteHistoryItem(historyItem, position)}
        .setNegativeButton("Cancel", null)
        .create()
        .show()}

```

Gambar 3.27 Kode Program untuk Menampilkan Konfirmasi Penghapusan Riwayat



```

override fun getItemCount(): Int = historyList.size

fun removeItem(position: Int) {
    if (position in 0 until historyList.size) {
        historyList.removeAt(position)
        notifyItemRemoved(position)
        notifyItemRangeChanged(position, historyList.size)}}

```

Gambar 3.28 Kode Program untuk Menghapus Item Riwayat dari Halaman



```

private fun toggleExpansion(holder: HistoryViewHolder, markdownText: String) {
    holder.isExpanded = !holder.isExpanded
    if (holder.isExpanded) {
        holder.diseaseInfoTextView.maxLines = Int.MAX_VALUE
        markdown?.setMarkdown(holder.diseaseInfoTextView, markdownText)
    } else {
        holder.diseaseInfoTextView.maxLines = 1
        holder.diseaseInfoTextView.text = "Tap to read disease information..."}

```

Gambar 3.29 Kode Program untuk Menampilkan Informasi Penyakit

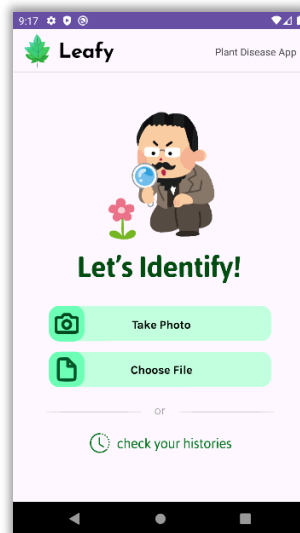
BAB IV

PROSEDUR PENGGUNAAN SISTEM

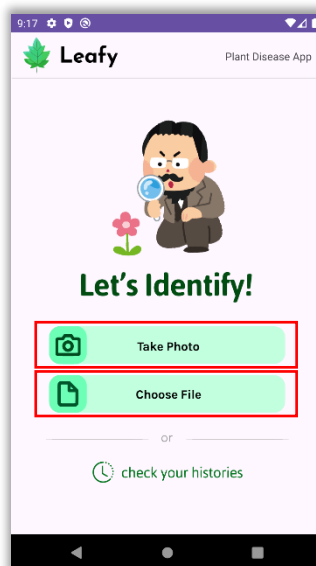
4.1 Prosedur Fitur Identifikasi

Berikut adalah prosedur untuk melakukan prediksi menggunakan aplikasi yang telah dikembangkan:

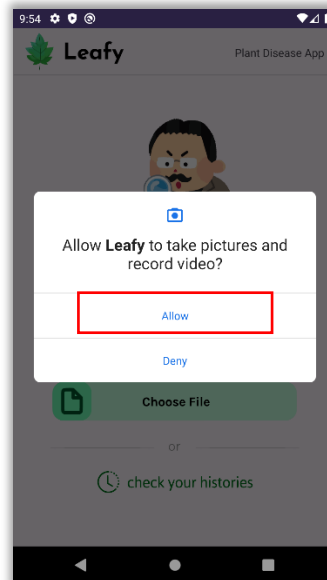
1. Buka aplikasi, maka tampilan halaman utama akan muncul.



2. Tekan tombol “*Take Photo*” atau “*Choose File*” yang ditandai dengan kotak merah untuk memulai proses identifikasi.



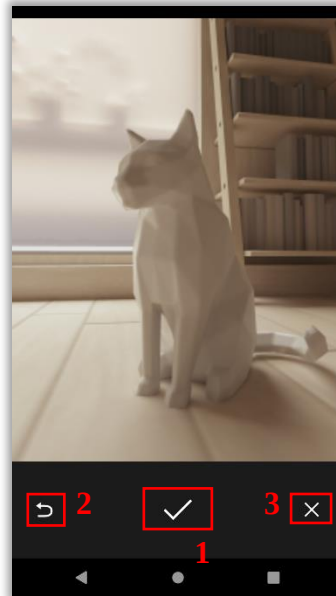
3. Apabila pengguna menekan tombol "*Take Photo*", akan muncul permintaan izin akses kamera jika ini adalah pertama kalinya aplikasi digunakan. Tekan tombol "*Allow*" sesuai petunjuk pada gambar yang ditandai dengan kotak merah.



4. Setelah tampilan kamera muncul, pengguna dapat mengambil gambar dengan menekan tombol kamera (kotak merah nomor 1), mengganti kamera (kotak merah nomor 2), mengaktifkan *grid* (kotak merah nomor 3), atau mengaktifkan *timer* (kotak merah nomor 4).



5. Setelah gambar diambil atau dipilih, pengguna dapat melakukan beberapa tindakan: mengonfirmasi gambar dengan menekan tombol centang (kotak merah nomor 1), membatalkan proses dengan menekan tombol kembali (kotak merah nomor 2), atau keluar dari halaman pengambilan gambar dengan menekan tombol X (kotak merah nomor 3).



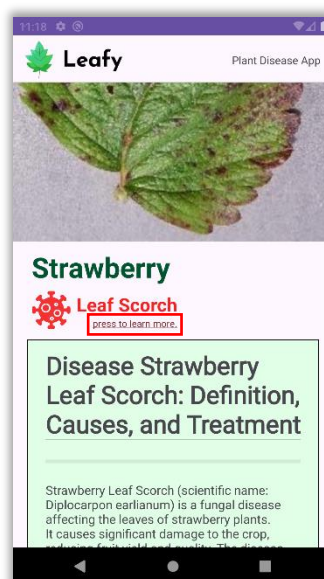
6. Jika sebelumnya pengguna memilih “Choose File”, Galeri akan terbuka, dan pengguna dapat memilih foto yang ingin diidentifikasi.



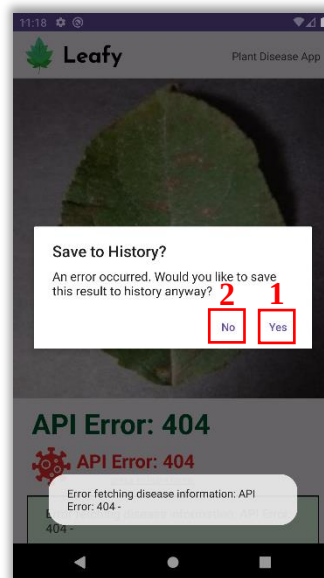
7. Setelah mengambil gambar atau memilih dari Galeri, pengguna akan masuk ke halaman *crop*. Di sini, pengguna dapat mengatur ukuran gambar (kotak merah 1) atau rotasi (kotak merah 2). Setelah puas, tekan tombol centang (kotak merah 3) untuk melanjutkan identifikasi, atau tombol X (kotak merah 4) untuk membatalkan.



8. Setelah beberapa saat, hasil prediksi akan muncul. Pengguna dapat menggulir halaman untuk melihat deskripsi penyakit dan menekan teks yang ditandai dengan kotak merah untuk informasi lebih lanjut.



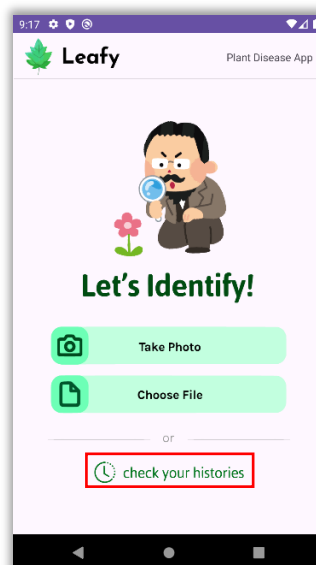
9. Jika terjadi masalah pada *server*, pengguna akan melihat hasil seperti gambar berikut dan dapat memilih untuk menyimpan (kotak merah 1) atau tidak menyimpan pesan *error* tersebut (kotak merah 2).



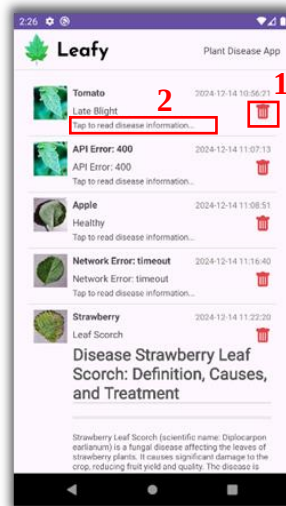
4.2 Prosedur Fitur Riwayat

Berikut adalah prosedur untuk melakukan prediksi menggunakan aplikasi yang telah dikembangkan:

1. Di halaman utama, tekan tombol “*Check Your Histories*” (kotak merah) untuk membuka halaman riwayat.



2. Pada halaman riwayat, pengguna dapat melihat entri identifikasi sebelumnya yang mencakup gambar, jenis tanaman, penyakit, serta tanggal dan waktu. Setiap entri memiliki dua tombol: tombol *Delete* (ikon tempat sampah merah, pada kotak merah 1) untuk menghapus entri, dan tombol "*Tap to Read Disease Information*" (pada kotak merah 2) untuk menampilkan informasi penyakit.



3. Jika pengguna ingin menghapus entri, akan muncul pesan konfirmasi untuk memastikan tindakan tersebut. Pengguna dapat memilih "*Delete*" (kotak merah 1) untuk menghapus atau "*Cancel*" (kotak merah 2) untuk membatalkan.

